Module 08

Introduction to SQL/1

Milav Dabgar

www.milav.in

March 17, 2026

Module Recap

- Introduced relational algebra
- Familiarized with the operators of relational algebra

Module Objectives

- To understand relational query language
- To understand data definition and basic query structure

Module Outline

- History of SQL
- Data Definition Language (DDL)
- Data Manipulation Language (DML): Query Structure

History of SQL

History of Query Language

- IBM developed Structured English Query Language (**SEQUEL**) as part of System R project. Renamed Structured Query Language (SQL; pronounced still as SEQUEL)
- ANSI and ISO standard SQL:

SQL-86	First formalized by ANSI
SQL-89	+ Integrity Constraints
SQL-92	Major revision (ISO/IEC 9075 standard), De-facto Industry Standard
SQL:1999	+ Regular Expression Matching, Recursive Queries, Triggers, Support for Procedural and Control Flow Statements, Nonscalar types (Arrays), and Some OO features, Embedding SQL in Java (SQL/OLB), and Embedding Java in SQL (SQL/JRT)
SQL:2003	+ XML features (SQL/XML), Window Functions, Standardized Sequences, and Columns with Auto-generated Values

History of Query Language (cont.)

SQL:2006	+ Ways of importing and storing XML data in an SQL database, manipulating it, and publishing both XML and conventional SQL-data in XML form
SQL:2008	Legalizes `ORDER BY` outside Cursor Definitions + `INSTEAD OF` Triggers, `TRUNCATE` Statement, and `FETCH` Clause
SQL:2011	+ Temporal Data (`PERIOD FOR`), Enhancements for Window Functions and `FETCH` Clause
SQL:2016	+ Row Pattern Matching, Polymorphic Table Functions, and JSON
SQL:2019	+ Multidimensional Arrays (MDarray type and operators)

History of Query Language (2): Compliance

- SQL is the de facto industry standard today for relational or structured data systems
- Commercial systems as well as open systems may be fully or partially compliant to one or more standards from SQL-92 onward
- Not all examples here may work on your particular system. Check your system's SQL documentation!

History of Query Language (3): Alternatives

- There aren't any alternatives to SQL for speaking to relational databases (that is, SQL as a protocol), but there are many alternatives to writing SQL in the applications
- These alternatives have been implemented in the form of frontends for working with relational databases.
- Some examples of a frontend include:
  - **SchemeQL** and **CLSQL**, which are probably the most flexible, owing to their Lisp heritage
  - **LINQ** (in .Net)
  - **ScalaQL** and **ScalaQuery** (in Scala)
  - **SqlStatement**, **ActiveRecord** and many others in Ruby
  - **HaskellDB**
  - ...the list goes on for many other languages.

History of Query Language (4): Derivatives

- There are several query languages that are derived from or inspired by SQL. Of these, the most popular and effective is **SPARQL**.
- **SPARQL** (pronounced *sparkle*, a recursive acronym for SPARQL Protocol and RDF Query Language) is an RDF query language
  - A semantic query language for databases - able to retrieve and manipulate data stored in Resource Description Framework (RDF) format.
  - It has been standardized by the W3C Consortium as key technology of the semantic web
  - Versions: SPARQL 1.0 (January 2008) and SPARQL 1.1 (March, 2013)
  - Used as the query languages for several NoSQL systems - particularly the Graph Databases that use RDF as store

Data Definition Language (DDL)

Data Definition Language (DDL)

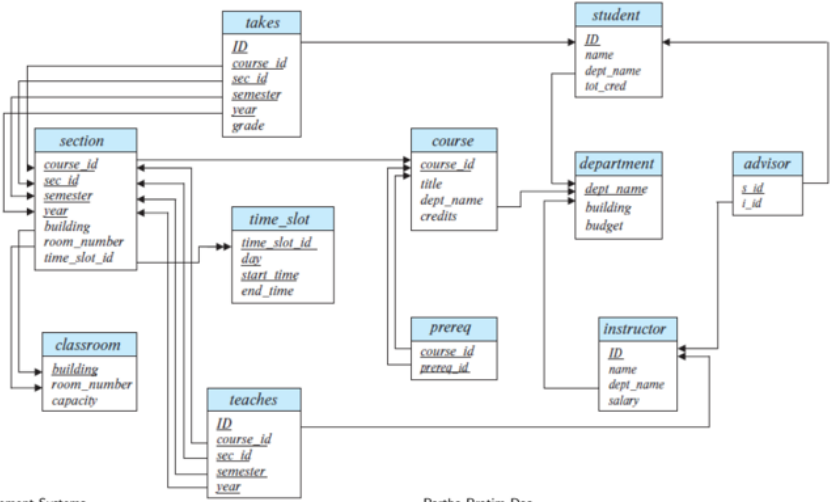
The SQL data-definition language (DDL) allows the specification of information about relations, including:

- The Schema for each Relation
- The Domain of values associated with each Attribute
- Integrity Constraints
- And also other information such as:
  - The set of Indices to be maintained for each relation
  - Security and Authorization information for each relation
  - The Physical Storage Structure of each relation on disk

Domain Types in SQL

- `char(n)`: Fixed length character string, with user-specified length *n*
- `varchar(n)`: Variable length character strings, with user-specified maximum length *n*
- `int`: Integer (a finite subset of the integers that is machine-dependent)
- `smallint(n)`: Small integer (a machine-dependent subset of the integer domain type)
- `numeric(p, d)`: Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (e.g., `numeric(3, 1)` allows 44.5 but not 444.5 or 0.32)
- `real, double precision`: Floating point and double-precision floating point numbers
- `float(n)`: Floating point number, with user-specified precision of at least *n* digits

Schema Diagram for University Database



Database Management Systems Partha Pratim Das

Create Table Construct

- An SQL relation is defined using the `create table` command:

```
create table r (
    A1 D1,
    A2 D2,
    ...,
    An Dn,
    (integrity-constraint_1),
    ...,
    (integrity-constraint_k)
);
```

- `r` is the name of the relation
- each `A_i` is an attribute name in the schema of relation `r`
- `D_i` is the data type of values in the domain of attribute `A_i`

Create Table Construct (2)

```
create table instructor (
    ID          char(5),
    name        varchar(20),
    dept_name   varchar(20),
    salary      numeric(8,2)
);
```

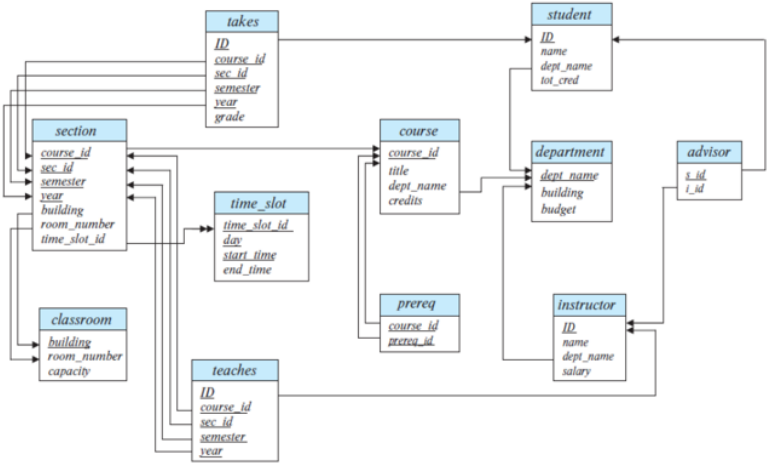
Create Table Construct (3): Integrity Constraints

- `not null`
- `primary key` $(A_1, \ldots, A_n)$
- `foreign key` $(A_m, \ldots, A_n)$ `references r`

```
create table instructor (
    ID          char(5),
    name        varchar(20) not null,
    dept_name   varchar(20),
    salary      numeric(8,2),
    primary key (ID),
    foreign key (dept_name) references department
);
```

Primary key declaration on an attribute automatically ensures `not null`

University Schema



Create Table Construct (4): More Relations

```
create table student (
    ID          varchar(5),
    name        varchar(20) not null,
    dept_name   varchar(20),
    tot_cred    numeric(3, 0),
    primary key (ID),
    foreign key (dept_name) references department
);

create table takes (
    ID          varchar(5),
    course_id   varchar(8),
    sec_id      varchar(8),
    semester    varchar(6),
    year        numeric(4, 0),
    grade       varchar(2),
    primary key (ID, course_id, sec_id, semester, year),
    foreign key (ID) references student,
    foreign key (course_id, sec_id, semester, year) references section
);
```

Note: `sec_id` can be dropped from primary key above, to ensure a student cannot be registered for two sections of the

Update Tables

- **Insert (DML command)**

```
insert into instructor values ('10211', 'Smith', 'Biology', 66000);
```

- **Delete (DML command)** : Remove all tuples from the student relation

```
delete from student;
```

- **Drop Table (DDL command)**

```
drop table r;
```

- **Alter (DDL command)**

```
alter table r add A D;
```

  - Where `A` is the attribute name and `D` is the domain
  - All existing tuples in the relation are assigned `null` for the new attribute

```
alter table r drop A;
```

  - Dropping of attributes is not supported by many databases

Data Manipulation Language (DML): Query Structure

Basic Query Structure

- A typical SQL query has the form:

$$\begin{aligned} &\texttt{select } A_1, A_2, \ldots, A_n \\ &\texttt{from } r_1, r_2, \ldots, r_m \\ &\qquad\qquad \texttt{where } P \end{aligned}$$

- A_i represents an attribute from r_i's
- r_i represents a relation
- P is a predicate
- The result of an SQL query is a **relation**

Select Clause

- The `select` clause lists the attributes desired in the result of a query
- Corresponds to the projection operation of the relational algebra
- Example: find the names of all instructors:

```
select name
from instructor;
```

- **NOTE**: SQL names are case insensitive (that is, you may use upper-case or lower-case letters)
- `Name` $\equiv$ `NAME` $\equiv$ `name`
- Some people use upper case wherever we use bold font

Select Clause (2)

- SQL allows duplicates in relations as well as in query results!!!
- To force the elimination of duplicates, insert the keyword `distinct` after `select`
- Find the department names of all instructors, and remove duplicates:

```
select distinct dept_name
from instructor;
```

- The keyword `all` specifies that duplicates should not be removed

```
select all dept_name
from instructor;
```

Select Clause (3)

- An asterisk in the select clause denotes `all` attributes:

```
select *
from instructor;
```

- An attribute can be a literal with no `from` clause:

```
select '437';
```

- Results is a table with one column and a single row with value `'437'`
- Can give the column a name using: `select '437' as FOO`
- An attribute can be a literal with `from` clause:

```
select 'A' from instructor;
```

- Result is a table with one column and N rows (number of tuples in the instructors table), each row with value `'A'`

Select Clause (4)

- The `select` clause can contain arithmetic expressions involving the operation +, −, ∗, and /, and operating on constants or attributes of tuples
- The query:

```
select ID, name, salary/12
from instructor;
```

- Would return a relation that is the same as the `instructor` relation, except that the value of the attribute `salary` is divided by 12
- Can rename `salary/12` using the `as` clause:

```
select ID, name, salary/12 as monthly_salary
from instructor;
```

Where Clause

- The `where` clause specifies conditions that the result must satisfy
- Corresponds to the selection predicate of the relational algebra
- To find all instructors in Comp. Sci. dept:

```
select name
from instructor
where dept_name = 'Comp. Sci.';
```

- Comparison results can be combined using the logical connectives `and`, `or`, and `not`
- Comparisons can be applied to results of arithmetic expressions

```
select name
from instructor
where dept_name = 'Comp. Sci.' and salary > 80000;
```

From Clause

- The `from` clause lists the relations involved in the query
- Corresponds to the Cartesian product operation of the relational algebra
- Find the Cartesian product `instructor` $\times$ `teaches`:

```
select *
from instructor, teaches;
```

- Generates every possible instructor-teaches pair, with all attributes from both relations
- For common attributes (for example, `ID`), the attributes in the resulting table are renamed using the relation name (for example, `instructor.ID`)
- Cartesian product not very useful directly, but useful combined with `where`-clause condition

Cartesian Product - Illustration

instructor $\times$ **teaches**

Module Summary

- Introduced relational query language
- Familiarized with data definition and basic query structure

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.